

Firmware and Telemetry Software for Munir-3 CubeSat

Software Report

Andreas Sæveraas
September 2026

Table of contents

1 Abstract.....	3
2 Introduction.....	4
3 Theory.....	5
3.1 Data collection and buffering.....	5
3.2 Serial communication and binary packet framing.....	5
3.3 Packet-based radio communication.....	6
4 Implementation.....	7
4.1 STM32 initialization and data collection.....	9
4.2 Telemetry packet and buffering.....	9
4.3 STM32-to-AVR communication.....	10
4.4 AVR and CC1200 communication.....	11
4.5 Ground station receiver.....	13
5 Results.....	15
6 Discussion.....	16
7 Conclusion.....	17

1 Abstract

Munir-3 is a 1U CubeSat developed to investigate plasma conditions in low Earth orbit and demonstrate an antenna integrated into the satellite structure. This report describes the software used to collect plasma-probe and auxiliary sensor data, package the measurements, transmit them from the satellite, and receive and store them at the ground station.

The onboard firmware was divided between an STM32 main controller and an AVR communication controller. The STM32 collected data from four plasma-probe ADCs together with orientation, acceleration, magnetic-field, and temperature measurements. The data were packed into fixed binary packets and sent over UART to the AVR, which controlled a CC1200 radio over SPI and transmitted the packets to the ground station.

The complete data path was successfully tested before launch, from measurement collection to storage on the ground station computer. No telemetry was received after launch, so the operation of the software in orbit could not be verified.

2 Introduction

Munir-3 is a 1U CubeSat carrying plasma probes intended to measure plasma conditions in low Earth orbit. The satellite also collects orientation, acceleration, magnetic-field, and temperature data. This report focuses on the firmware and communication system used to collect these measurements and transfer them to a ground station computer.

An STM32F373 was used as the main onboard controller, while a separate AVR32DA28 controlled the CC1200 radio transceiver. Telemetry therefore passed from the STM32 to the AVR before transmission. The ground station used another AVR32DA28 and CC1200 to receive the packets and forward them to a computer.

The main purpose of the software was to move measurement data reliably through the complete system. The report therefore covers data collection, packet construction, communication between the onboard controllers, radio control, and receiving the data at ground station. The complete chain worked during pre-flight testing, but no data was received after launch.

3 Theory

This section gives the background needed to understand the software implementation. The main topics are data collection and buffering, serial communication and packet framing, and packet-based radio communication. These concepts are used throughout the software to move measurement data from the onboard sensors to the ground station.

3.1 Data collection and buffering

Measurements that change over time should be collected at regular intervals. This is especially important for the plasma probes, since changes in the sampling interval can make the measured data harder to interpret.

A hardware timer can be used to keep the sampling interval regular. The timer runs independently of the main program and can signal when a new measurement should be collected. The actual communication with the sensors can then be handled in the main program instead of inside the timer interrupt. This keeps the interrupt short and avoids spending too much time inside it.

Data collection and transmission do not always happen at the same time. Two buffers can therefore be used so that one buffer is filled with new measurements while the other contains a completed packet waiting to be transmitted. This allows data collection to continue while an earlier packet is being sent.

DMA can also reduce the amount of work done by the processor during communication. Once a DMA transfer is started, it can move data directly between memory and a peripheral such as UART without the processor handling every byte.

3.2 Serial communication and binary packet framing

UART, SPI, and I2C are used to transfer data between the controllers and peripherals in the system. UART provides asynchronous serial communication, SPI provides faster synchronous communication using a shared clock, and I2C allows several addressed devices to share the same bus.

UART sends data as a continuous stream of bytes, so the receiver needs a way to determine where each packet starts. Synchronization bytes can be placed before each packet so that the receiver can find the beginning of the packet. A length field can then be used to tell the receiver how many bytes belong to it.

The telemetry is transmitted in binary form. Each measurement is stored directly using a fixed data type and size instead of first being converted to text. This keeps the packets

relatively small and reduces the amount of data that needs to be transmitted. The sender and receiver must therefore use the same packet structure and interpret the fields in the same way.

3.3 Packet-based radio communication

A packet-based radio transceiver sends data as separate radio packets. The transmitter and receiver must use matching radio settings so that the receiver can correctly detect and decode the transmitted data.

A synchronization word is used to identify the beginning of a packet. Packet-length information tells the receiver how much data belongs to the packet, while a cyclic redundancy check (CRC) can be used to detect errors that occur during transmission. A packet with an incorrect CRC can then be rejected.

Radio transceivers often use transmit and receive FIFOs (First-In, First-Out buffers). Before transmission, the controller writes the packet data to the transmit FIFO. At the receiving side, received data are stored in the receive FIFO until the controller reads them.

The radio can also provide information such as RSSI, which gives an indication of the received signal strength. The firmware should also handle problems such as timeouts, FIFO errors, and unexpected radio states so that communication errors do not stop the system from operating correctly.

4 Implementation

The Munir-3 software was divided between an STM32F373 main controller, an AVR32DA28 communication controller, an AVR32DA28 ground station controller, and the ground station PC software. [Table 1](#) summarizes the main devices and interfaces.

Table 1. Main hardware components and communication interfaces used in the system.

Function	Device	Interface
Main onboard controller	STM32F373 [3] , [4]	—
Communication controller	AVR32DA28 [5]	UART/SPI
Ground station controller	AVR32DA28 [5]	SPI/UART
Plasma-probe conversion	4 × ADS7279 [6]	SPI
IMU	BNO055 [1]	I2C
Radio transceiver	CC1200 [2]	SPI
Temperature sensor	LM92 [7]	I2C

The overall software and telemetry architecture is shown in [Figure 1](#). The STM32 collects the sensor data and assembles it into telemetry packets, while the onboard AVR handles communication with the radio. At the ground station, another AVR receives the radio packets and forwards them to a computer for storage.

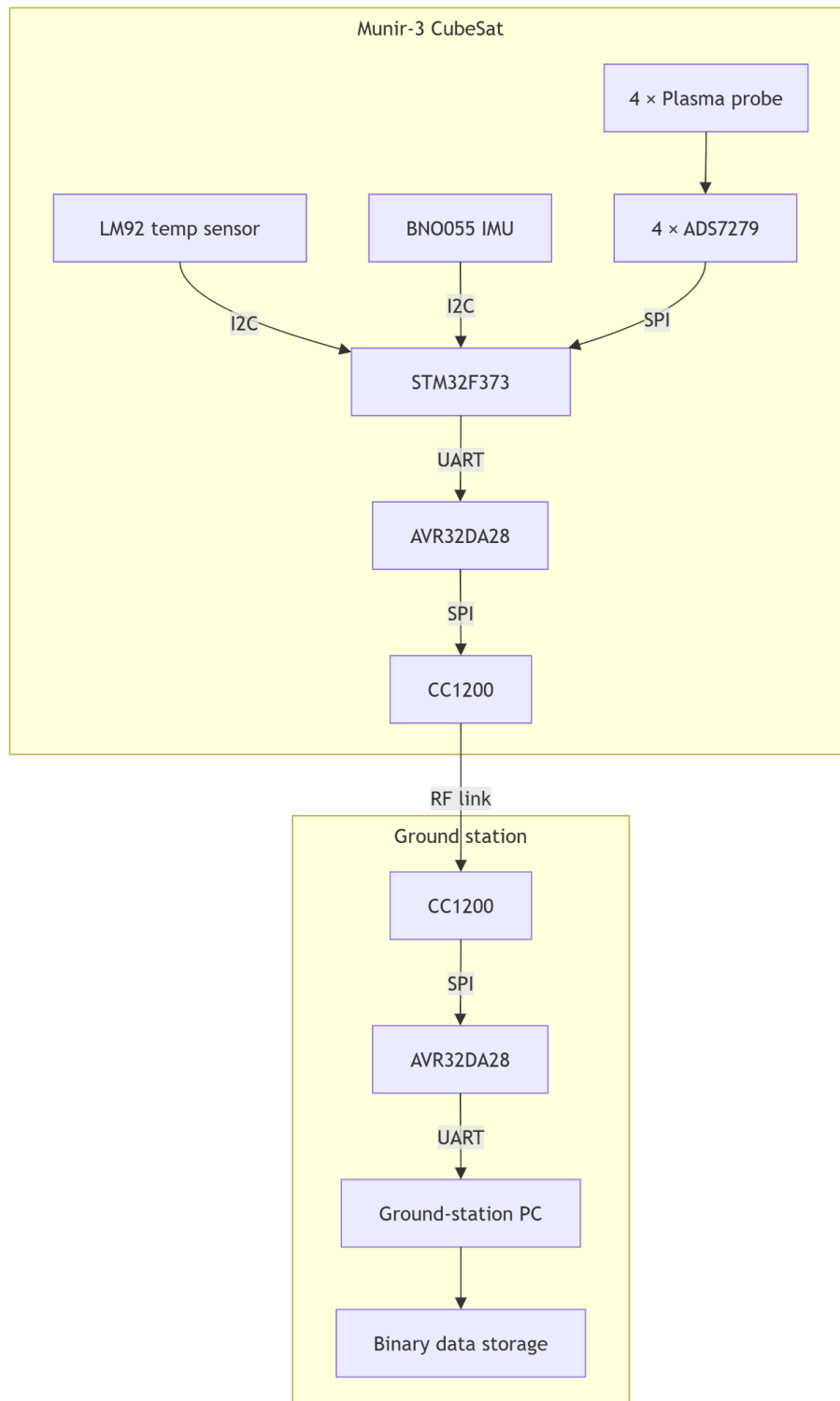


Figure 1. Overall software and telemetry architecture of Munir-3.

4.1 STM32 initialization and data collection

At startup, the STM32F373 initializes the GPIO, DMA, I2C, UART, SPI, and timer peripherals [3], [4]. The BNO055 is configured in NDOF mode [1], and the four ADS7279 ADCs, one for each plasma probe, are initialized.

Before measurement begins, the STM32 performs the deployment sequence. The firmware waits 15s, activates the burn-wire output for 15s, and then disables it before entering the main measurement loop.

The STM32 uses its 8 MHz HSI oscillator as the system clock [3]. The timers are used for three different purposes. TIM2 channel 2 is configured in PWM mode to generate PWM_FL_CLK, a clock signal for the external plasma-probe filter. With a timer period of 2207 and compare value of 1103, this produces a square wave of approximately 3.62 kHz with a 50% duty cycle. TIM3 provides the timing for plasma-probe sampling. With a prescaler of 7 and a period of 999, it generates an interrupt every 1 ms. The firmware requests an ADC read every fourth interrupt, resulting in a collection interval of approximately 4 ms.

Each collection reads all four ADS7279 devices and produces four 14-bit measurements [6], stored as 16-bit values in the packet. The packet contains 48 ADC values, which corresponds to twelve four-channel collections:

$$12 \times 4 \text{ ms} = 48 \text{ ms}$$

This means that the ADC data in each telemetry packet represents 48 ms of plasma-probe measurements. The timer interrupt only signals that a collection is needed. The actual SPI communication with the ADCs is handled in the main program.

Auxiliary measurements are collected once for each packet. The BNO055 provides Euler orientation, linear acceleration, magnetic-field measurements, and temperature [1]. Its temperature output has a resolution of 1°C [1]. A second temperature value is read from a LM92 temperature sensor, which has a much finer resolution of 0.0625°C [7].

TIM5 runs continuously as a 32-bit counter with a resolution of about 1 μ s. Its value is copied into the packet when the packet is initialized. The timestamp therefore marks the start of the packet rather than the time of each individual ADC measurement. Since the ADC measurements are collected at known 4 ms intervals, the approximate collection time of each measurement can be determined from the packet timestamp and its position in the packet.

4.2 Telemetry packet and buffering

The measurements are stored in a packed binary structure that is transmitted directly from memory. The packet format is shown in [Table 2](#).

Table 2. Structure of the STM32 telemetry packet.

Field	Data type	Size [bytes]	Description
Length	uint8_t	1	Number of bytes after the length field
Timestamp	uint32_t	4	TIM5 counter when the packet is initialized, 1 μ s resolution
Cycle	uint16_t	2	Legacy field from an earlier packet format, not used in final data processing
Euler	int16_t[3]	6	Three-axis orientation
Acceleration	int16_t[3]	6	Three-axis linear acceleration
Magnetometer	int16_t[3]	6	Three-axis magnetic-field measurement
Temperature 1	uint16_t	2	LM92 temperature
Temperature 2	int8_t	1	BNO055 temperature
ADC payload	uint16_t[48]	96	48 ADS7279 plasma-probe measurements, stored as 16-bit values

The complete packet is 124 bytes. The first byte contains the payload length and is set to 123.

Because the packed structure is transmitted directly from memory, the receiving software must use the same field sizes, order, and byte representation.

Two packet buffers are used. One is filled with new ADC measurements while the other can hold a completed packet waiting to be transmitted. After 48 ADC values have been collected, the completed buffer is marked as ready, and collection continues in the other buffer.

4.3 STM32-to-AVR communication

Completed packets are sent from the STM32F373 to the onboard AVR32DA28 over USART2. Since UART provides a continuous stream of bytes, the AVR needs a way to detect where each packet begins. Two synchronization bytes, 0xAA 0x55, are therefore added before every packet.

The complete serial frame is:

0xAA 0x55 | length | telemetry payload

The STM32 sends a 124-byte packet consisting of a one-byte length field and 123 bytes of telemetry. The AVR uses the synchronization bytes to locate the start of the packet and the length field to determine its size. Before radio transmission, the STM32 length byte is removed, leaving 123 bytes of telemetry as the radio payload.

USART2 is configured for 230400 baud, 8 data bits, 1 stop bit, no parity, and no hardware flow control. UART transmission uses DMA. A flag shows when the UART is busy, and the transmission-complete callback clears the flag when the transfer is finished.

4.4 AVR and CC1200 communication

The onboard AVR32DA28 controls the CC1200 radio over SPI [\[5\]](#). During initialization, it writes a predefined set of register values to the radio. Separate settings are used for the transmitter and receiver. The main radio settings are shown in [Table 3](#).

Table 3. Main CC1200 radio settings

Parameter	Setting
Carrier frequency	436 MHz
Modulation	2-GFSK
Data rate	about 40 kbit/s
Frequency deviation	about 25 kHz
Receiver channel bandwidth	about 208 kHz
Synchronization word	0x55557A0E
Packet length mode	Variable length
Crystal frequency	40 MHz
CRC	CRC-16 enabled

The driver can read the configuration registers back and compare them with the expected values. For transmission, the AVR writes the packet length and telemetry data to the CC1200 transmit FIFO and issues the STX command [2]. It then monitors the radio until transmission is complete.

The driver handles transmission timeouts, FIFO errors, and unexpected radio states. If a transmit FIFO error occurs, the FIFO is flushed before the driver returns an error.

The onboard AVR does little processing of the telemetry. Its main job is to receive data from the STM32 and pass it to the CC1200 for transmission. The onboard data flow is shown in [Figure 2](#).

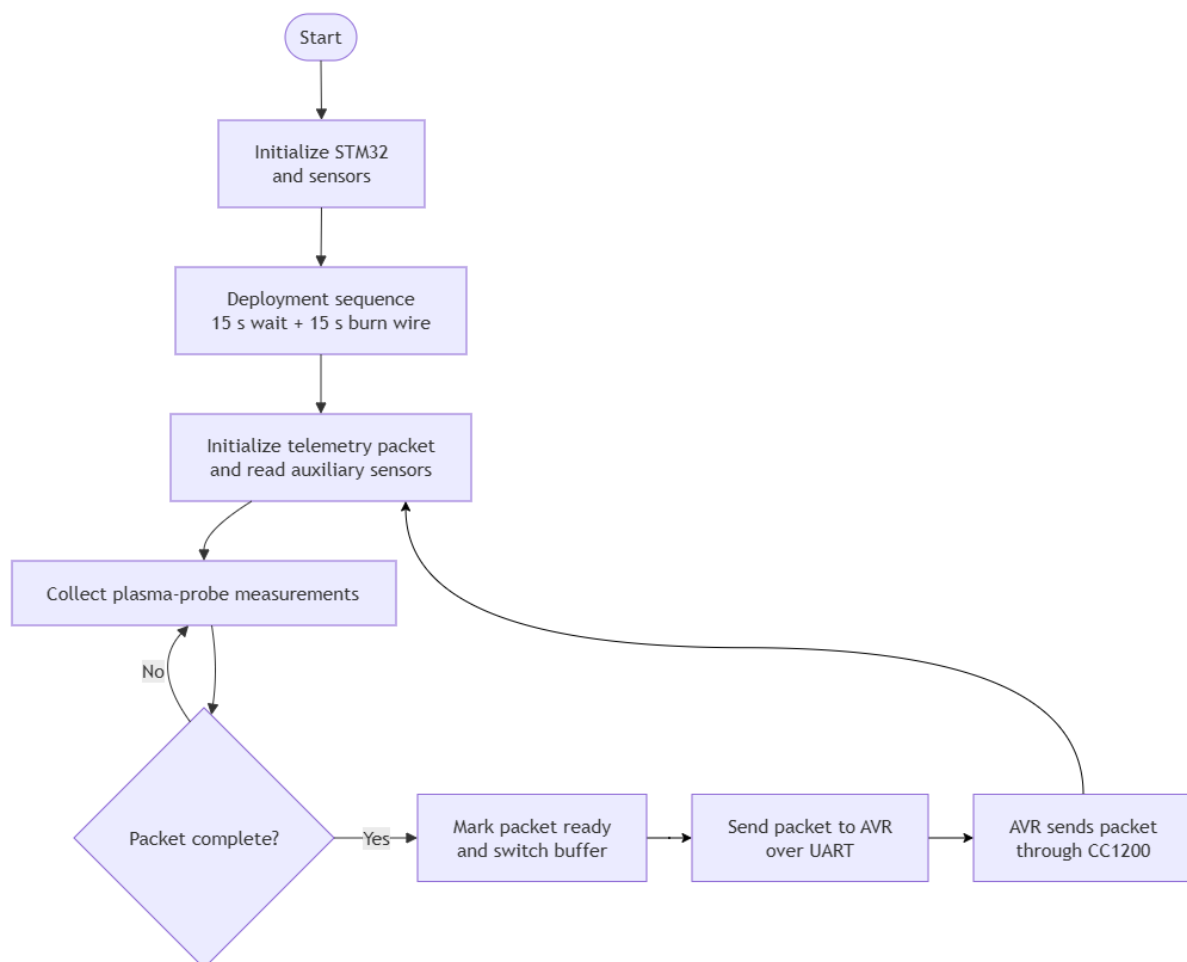


Figure 2. Simplified onboard data collection and telemetry flow

4.5 Ground station receiver

The ground station uses another AVR and CC1200 configured for receiving. The CC1200 waits in receive mode for a packet. When a packet arrives, the AVR reads the packet length and telemetry data from the receive FIFO [2]. It also monitors the radio for timeouts and FIFO errors.

After a successful reception, the AVR reads the CC1200 RSSI register [2] and forwards the raw RSSI value together with the telemetry. The data sent to the ground station computer have the format:

0xAA 0x55 | RSSI | 123-byte telemetry packet

The PC software reads the serial data and stores the binary stream directly. Decoding and analysis can then be done later without changing the original received data. The ground station data flow is shown in [Figure 3](#).

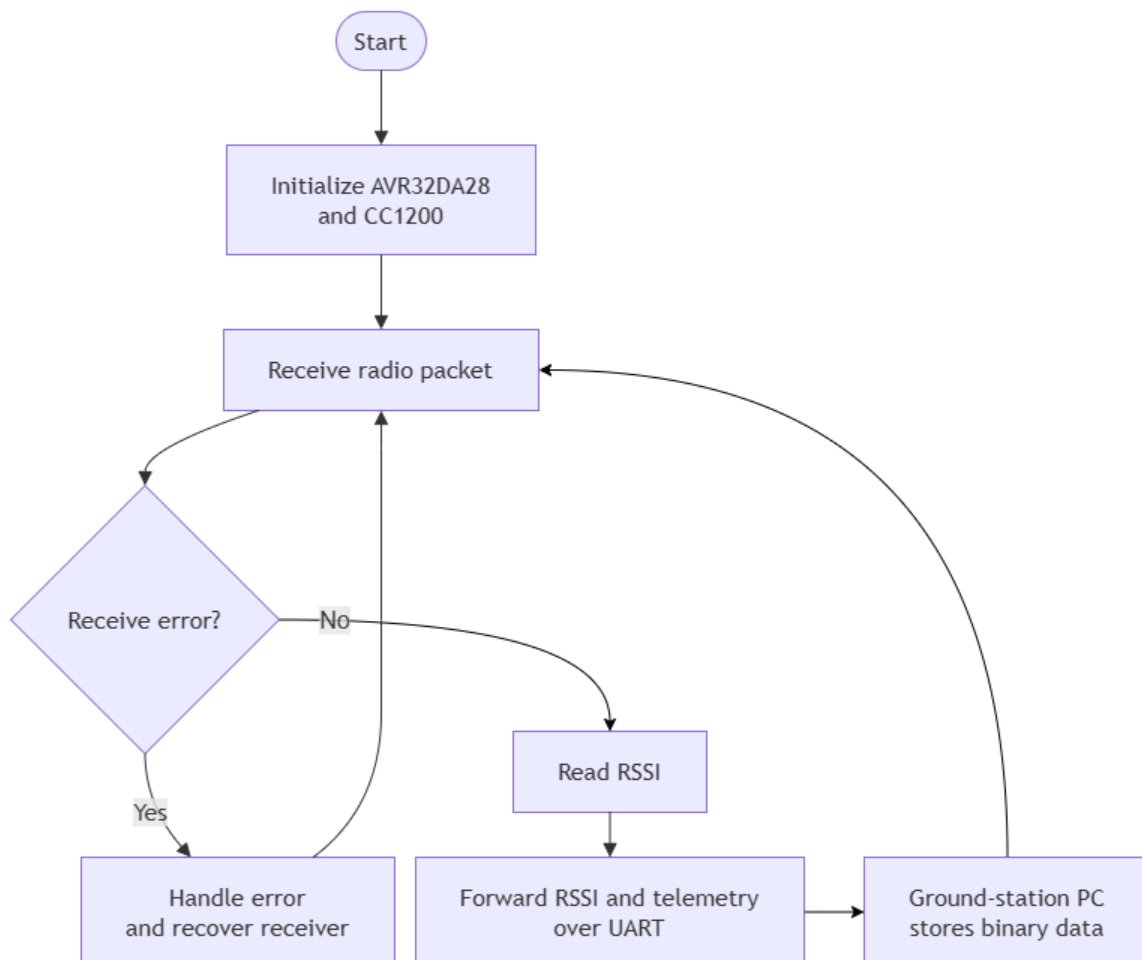


Figure 3. Simplified ground station data flow

5 Results

Pre-flight testing showed that the complete software and telemetry chain worked as intended. The STM32 collected data from the plasma-probe ADCs and auxiliary sensors, assembled the measurements into packets, and transferred them to the onboard AVR. The AVR then passed the telemetry to the CC1200 for radio transmission. At the ground station, the packets were received, forwarded to the computer, and stored successfully.

This testing verified the complete software data path from measurement collection to storage on the ground-station computer before launch.

No telemetry was received from the satellite after launch, so the operation of the software in orbit could not be verified.

After launch, the plasma probes were tested separately in a simulated plasma environment on the ground. This test was successful and showed that the probes themselves worked as intended under the test conditions, but it did not provide any information about the operation of the flight software in orbit.

6 Discussion

The software worked during ground testing, but the final architecture was more complicated than necessary. The STM32 already handled measurement collection and packet generation, while the AVR mainly acted as an intermediate controller for the CC1200. This added another communication link and increased the overall software complexity. A future design could allow the main controller to communicate directly with the radio.

The fixed binary packet format kept the transmitted data compact and simple to process. The packet also kept the legacy cycle field even though it was no longer used in final data processing. A future packet format could remove unused fields and include a format version.

The radio firmware handled basic errors such as timeouts and FIFO errors. The telemetry system only sent data in one direction and did not use acknowledgements or retransmission. This kept the system simpler and avoided slowing down the data stream, but meant that some packet loss had to be accepted.

The main limitation is that no telemetry was received after launch. Ground testing showed that the complete data path worked, but this does not prove that it operated correctly in orbit. A failure in the power system, processing, radio, antenna, or ground station could all lead to the same result, so the available information is not enough to identify the cause.

A future version could use a simpler onboard architecture. More time should also be given to testing the final flight firmware before launch.

7 Conclusion

This report described the firmware and telemetry software developed for the Munir-3 CubeSat. The STM32F373 collected measurements from four ADS7279 plasma-probe ADCs and the auxiliary sensors, packed the data into binary telemetry packets, and sent them to an AVR32DA28 communication controller. The AVR controlled the CC1200 radio, while another AVR32DA28 and CC1200 at the ground station received the telemetry and forwarded it to a computer for storage.

Pre-flight testing showed that the complete chain from measurement collection to ground station storage worked successfully.

No telemetry was received after launch, so it was not possible to confirm that the software worked correctly in orbit. The main lessons from the project were to keep the onboard architecture simple and allow more time for testing the final flight firmware.

References

- [1] Bosch Sensortec, [*BNO055 Intelligent 9-Axis Absolute Orientation Sensor – Datasheet.*](#)
- [2] Texas Instruments, [*CC120X Low-Power High Performance Sub-1 GHz RF Transceivers – User’s Guide, SWRU346B.*](#)
- [3] STMicroelectronics, [*STM32F373xx Datasheet, DocID022691, Rev. 7.*](#)
- [4] STMicroelectronics, [*STM32F37xxx Advanced Arm-Based 32-bit MCUs – Reference Manual, RM0313.*](#)
- [5] Microchip Technology, [*AVR32DA28/32/48\(S\) Complete Data Sheet, DS40002228.*](#)
- [6] Texas Instruments, [*ADS7279/ADS7280 Low-Power, 14-Bit, 1-MHz Single/Dual Unipolar Input Analog-to-Digital Converters with Serial Interface – Datasheet, SBAS436A.*](#)
- [7] Texas Instruments, [*LM92 \$\pm 0.33^{\circ}\text{C}\$ Accurate, 12-Bit + Sign Temperature Sensor and Thermal Window Comparator With Two-Wire Interface – Datasheet, SNIS110D, Rev. D.*](#)